Simply a Strong Variant

- Document number: P0093R0
- Date: 2015-09-24
- Project: ISO/IEC JTC1 SC22 WG21

- Reply-to: David Sankel david@stellarscience.com

Introduction

There is wide agreement that C++ needs a type-safe union type and the significant number of "variant" implementations in the wild confirms that it fills a practical need for many developers.[1] On the other hand, how exactly to implement a variant has been an item of much controversy of late. This is witnessed by many hundreds of variant emails on the standard library mailing list and the heated discussions invoked by related blog posts.[2][3] The danger, of course, with all the controversy is that consensus will not be achieved and that software developers will suffer the effects of widespread divergent implementations of a library considered to be an essential "vocabulary" type.

This paper is an attempt to bridge the divide by proposing a hitherto undiscussed option. To the author's surprise, this attempt at a compromise has the unusual properties of being both more flexible and more simple (in both specification and usage) than the competing alternatives.

In a nutshell, we are proposing a variant implementation that has the never-empty guarantee, preserves **strong** type safety, and gives implementers the flexibility to optimize their implementations based on the alternative types. As it turns out, this simple specification meets the needs of opposing camps and provides an interface that will allow more optimizations as language improvements come about in the future.

Design Considerations

Never empty

At the May 2015 meeting at Lenexa there was consensus that the variant type should model the mathematical concept of the discriminated union. This decision led to more consistency and provided guidance as to what the interface should look like. This is particularly true with regard to whether or not to allow repeated alternative types and how to handle conversions.

Although this consensus was achieved at Lenexa, there was a lingering contradiction between the attempt to model a discriminated union and the proposed implementation; namely, there is no mathematical equivalent of an "invalid" state. Although the committee took pains to limit the conditions required to get into the invalid sate, it nonetheless complicated the semantics of variant. Consider the following function declaration:

```
void f( std::variant<A,B,C> & );
```

`f` may or may not be able to properly handle an "invalid" variant. Although most functions with variant parameters do not *need* to handle an "invalid" variant, there are some that do and we need to distinguish between them. This can be somewhat obviated by having a coding standard that states "unless otherwise specified, variant parameters have a validity precondition", but the extra complication of the semantics still remains.

The variant of this proposal completely removes the "invalid" state and, thus, has no difficulties modeling a discriminated union.

Why Strong?

Peter Dimov pointed out that a variant with only basic exception safety is exceedingly difficult to use as a member variable of a class that requires strong exception guarantees. This fact is easy to see when comparing the requisite hurdles to those needed for other standard structures. The main difficulty is that a variant type is not guaranteed have a non-throwing move assignment operator and, thus, `swap` operation.

In effect, a class with strong exception guarantees with an arbitrary variant member will need to manually double buffer the variant. In the following example we have a class `C` that has strong exception safety guarantees and a `std::variant` member variable.

```
class C {
public:
  //...
```

```
  std::variant<A,B,C> getVariant() const;
  void modifyVariant();
private:
  bool m_firstVariantIsTheActiveOne = true;
  std::pair< std::variant<A,B,C>, std::variant<A,B,C> > m_variant;
};
```

The first thing to note is that the `getVariant()` function cannot return a `const &` since the "active" variant can change over time. The second thing to note is that we need a discriminator to maintain which of the two variants is active at any time. The implementations of the member functions follow:

```
std::variant<A,B,C> C::getVariant() const {
  return m_firstVariantIsTheActiveOne ? m_variant.first : m_variant.second;
}

void C::modifyVariant() {
  // make both variants the same
  if(m_firstVariantIsTheActiveOne)
    m_variant.second = m_variant.first;
  else
    m_variant.first = m_variant.second;
  m_firstVariantIsTheActiveOne = true;

  // Modify 'm_variant.second' in a way that could possibly throw an
  // exception.
  // ...

  m_firstVariantIsTheActiveOne = false;
}
```

This methodology differs from how we would implement a class with strong guarantees with any other standard type in the following ways:

- (basic guarantee styled) variant members restrict what the interface of the class's getters can look like.
- (basic guarantee styled) variant members require the class to have additional member variables.

Compare this to how modifications to a `std::vector` in a member function of a class with strong guarantees would be implemented. Special consideration only occurs in the member function itself.

```
void C::modifyVector() {
  std::vector<A> copy = m_vector;

  // Modify 'copy' in a way that could possibly throw an exception.
  // ...

  std::swap( copy, m_vector );
}
```

A variant with a strong exception safety guarantee (or more specifically a strong exception safety preserving guarantee) would not require any special considerations for this use case and that is what is being proposed in this paper.

Performance implications

A naive implementation of the proposed library would use double buffering as a means to provide strong exception safety guarantees. This implementation path has an important drawback: every variant instance would have a size that is twice that of the largest alternative type. None of the other variants proposed thus far have this drawback.

On closer inspection, however, one can see that not all variants would require double buffering. For example, if every alternative of a variant type has a non-throwing move constructor there is no need for double buffering. This is notably true for all built-in types. A non-double buffered variant could also be created if alternatives only supported non-throwing move assignment operations by using the stack for temporary values. This applies, or could apply, to most standard types.

While this proposal does not require implementations to take advantage of these optimizations, a quality implementation would make use of these strategies when possible, relieving the developer from these concerns.

One oft-cited concern is developers who are writing high-performance code. The underlying assumption is that they would not find double-buffering acceptable under any circumstances. The author speculates that these same developers are happy to trade program correctness for speed by avoiding exception safety issues by either disabling them or making use of `union`s in non-robust ways.

Even those developers, however, are supported by this proposal. They can make use of the `noexcept` keyword on the possibly exception-throwing move-constructors of the alternatives they use in the variant to ensure double-buffering is disabled on quality implementations. One may claim that these extra steps would be tedious, but the author of this paper argues that this is a feature; writing non-robust code should be made tedious by design.

Surprise: a future proof interface

Another, perhaps surprising, benefit of this proposal is that `variant` implementations can improve in performance as C++ gains more relevant features without resorting to changing the interface in possibly breaking ways. Transactional memory, for example, may one day allow all variants to avoid double buffering while always maintaining the strong exception safety guarantee. Other variant proposals that have either an explicit or implicit empty state are, in essence, pessimizing the public interface with regard to future improvements in the language.

Conclusion

The need for a standard variant type is clear. This paper proposes a variant that is simply defined, is easy to use, and meets a wide variety of needs.

---

1. Variant: a type-safe union (v4). N4542↩
2. Standardizing Variant: Difficult Decisions↩
3. A variant for the everyday Joe↩